Exercise5_1

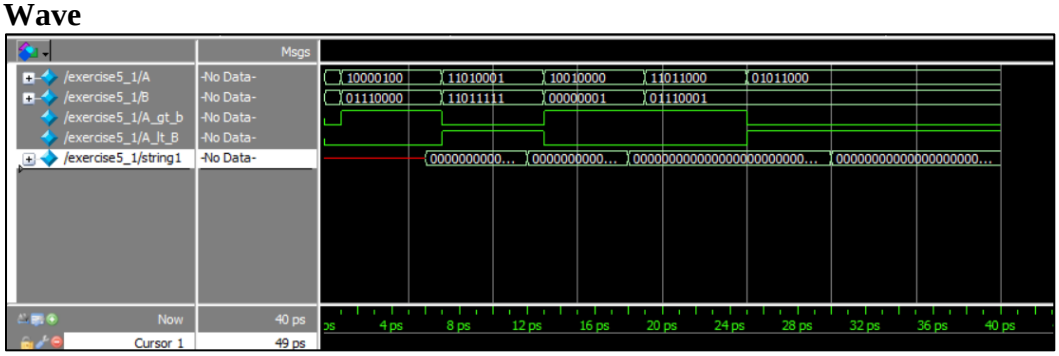
HDL Code

```
module BitComparator(output A_GT_B, A_LT_B, input [7:0] A, B);
    assign A_GT_B = (A > B);
    assign A_LT_B = (A < B);
endmodule

module exercise5_1;
    reg [7:0] A, B;
    wire A_gt_b, A_lt_B;
    reg [255:0] string1;
    BitComparator test(A_gt_b, A_lt_B, A, B);

    initial begin
        A = 8'b00000000; B = 8'b00000000;
        $display("A \t\t\t\t B \t\t\t\t Remarks");

        #1 A = 8'b10000100; B = 8'b01110000;
        #5 if (A == B) string1 = "Equal"; else if (A_gt_b) string1 = "A is greater than B"; else if (A_lt_B) string1 = "A is less than B";
        $display("%b \t\t\t\t %b \t %s", A, B, string1);

        #1 A = 8'b11010001; B = 8'b11011111;
        #5 if (A == B) string1 = "Equal"; else if (A_gt_b) string1 = "A is greater than B"; else if (A_lt_B) string1 = "A is less than B";
        $display("%b \t\t\t\t %b \t %s", A, B, string1);

        #1 A = 8'b10010000; B = 8'b00000001;
        #5 if (A == B) string1 = "Equal"; else if (A_gt_b) string1 = "A is greater than B"; else if (A_lt_B) string1 = "A is less than B";
        $display("%b \t\t\t\t %b \t %s", A, B, string1);

        #1 A = 8'b11011000; B = 8'b01110001;
        #5 if (A == B) string1 = "Equal"; else if (A_gt_b) string1 = "A is greater than B"; else if (A_lt_B) string1 = "A is less than B";
        $display("%b \t\t\t\t %b \t %s", A, B, string1);

        #1 A = 8'b01011000; B = 8'b01110001;
        #5 if (A == B) string1 = "Equal"; else if (A_gt_b) string1 = "A is greater than B"; else if (A_lt_B) string1 = "A is less than B";
        $display("%b \t\t\t\t %b \t %s", A, B, string1);

        #10 $finish;
    end
endmodule
```

Transcript

```
VSIM 16> run
# GetModuleFileName: The specified module could not be found.
#
#
# A                               B                               Remarks
# 10000100                        01110000                A is greater than B
# 11010001                        11011111                   A is less than B
# 10010000                        00000001                A is greater than B
# 11011000                        01110001                A is greater than B
# 01011000                        01110001                   A is less than B
# ** Note: $finish    : C:/intelFPGA_lite/20.1/Exercise5_1.v(36)
#    Time: 40 ps  Iteration: 0  Instance: /exercise5_1
# 1
# Break in Module exercise5_1 at C:/intelFPGA_lite/20.1/Exercise5_1.v line 36
```

Wave



Exercise5_2

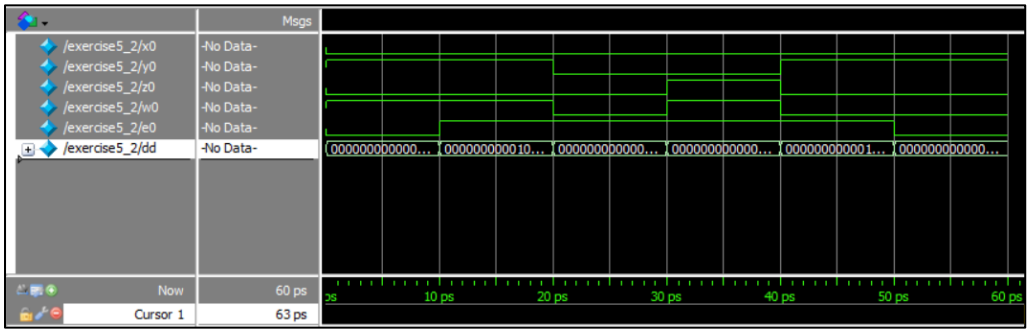
HDL Code

```
module decoder(x,y,z,w,e,d);
    input w, x, y, z, e;
    output [15:0] d;

    assign d[0] = (~x) & (~y) & (~z) & (~w) & e;
    assign d[1] = (~x) & (~y) & (~z) & w & e;
    assign d[2] = (~x) & (~y) & z & (~w) & e;
    assign d[3] = (~x) & (~y) & z & w & e;
    assign d[4] = (~x) & y & (~z) & (~w) & e;
    assign d[5] = (~x) & y & (~z) & w & e;
    assign d[6] = (~x) & y & z & (~w) & e;
    assign d[7] = (~x) & y & z & w & e;
    assign d[8] = x & (~y) & (~z) & (~w) & e;
    assign d[9] = x & (~y) & (~z) & w & e;
    assign d[10] = x & (~y) & z & (~w) & e;
    assign d[11] = x & (~y) & z & w & e;
    assign d[12] = x & y & (~z) & (~w) & e;
    assign d[13] = x & y & (~z) & w & e;
    assign d[14] = x & y & z & (~w) & e;
    assign d[15] = x & y & z & w & e;
endmodule

module exercise5_2();
    reg x0, y0, z0, w0, e0;
    wire [15:0] dd;

    decoder s(x0, y0, z0, w0, e0, dd);

    initial begin
        e0 = 0; x0 = 0; y0 = 1; z0 = 0; w0 = 1;
        $display("Time\te0\tx0\ty0\tz0\tw0\td[15:0]");
        #10 e0 = 1; $display("%0t\t%d\t%d\t%d\t%d\t%d\t%b", $time, e0, x0, y0, z0, w0, dd);
        #10 x0 = 0; y0 = 0; z0 = 0; w0 = 0; $display("%0t\t%d\t%d\t%d\t%d\t%d\t%b", $time, e0, x0, y0, z0, w0, dd);
        #10 x0 = 0; y0 = 0; z0 = 1; w0 = 1; $display("%0t\t%d\t%d\t%d\t%d\t%d\t%b", $time, e0, x0, y0, z0, w0, dd);
        #10 x0 = 0; y0 = 1; z0 = 0; w0 = 0; $display("%0t\t%d\t%d\t%d\t%d\t%d\t%b", $time, e0, x0, y0, z0, w0, dd);
        #10 e0 = 0; $display("%0t\t%d\t%d\t%d\t%d\t%d\t%b", $time, e0, x0, y0, z0, w0, dd);
        #10 $finish;
    end
endmodule
```

Transcript

```
VSIM 21> run
# GetModuleFileName: The specified module could not be found.
#
#
# Time   e0      x0      y0      z0      w0      d[15:0]
# 10     1       0       1       0       1       0000000000000000
# 20     1       0       0       0       0       0000000000100000
# 30     1       0       0       1       1       0000000000000001
# 40     1       0       1       0       0       0000000000001000
# 50     0       0       1       0       0       0000000000010000
# ** Note: $finish    : C:/intelFPGA_lite/20.1/Exercise5_2.v(37)
#    Time: 60 ps  Iteration: 0  Instance: /exercise5_2
# 1
# Break in Module exercise5_2 at C:/intelFPGA_lite/20.1/Exercise5_2.v line 37
```

Wave



Drill5_1

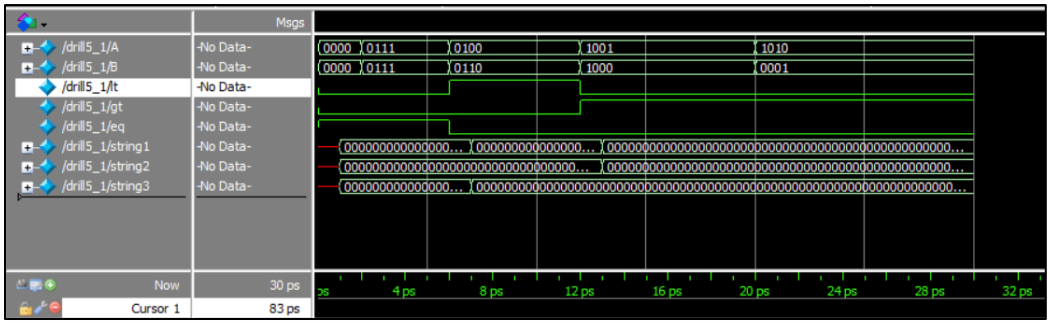
HDL Code

```
module magcom (input [3:0] A, B, output lt, gt, eq);
        assign lt = (A < B);
        assign gt = (A > B);
        assign eq = (A == B);
endmodule

module drill5_1;
        reg [3:0] A, B;
        wire lt, gt, eq;
        magcom tb(A, B, lt, gt, eq);
        reg [255:0] string1, string2, string3;

        initial begin
                $display("A B\tA<B\tA>B\tA==B");
                A = 4'b0; B = 4'b0; #1;
                if (lt==1) string1="true"; else string1="false";
                if (gt==1) string2="true"; else string2="false";
                if (eq==1) string3="true"; else string3="false";
                $display("%0d %0d\t%0s\t%0s\t%0s", A, B, string1, string2, string3);
                #1 A = 4'd7; B = 4'd7; #1;
                if (lt==1) string1="true"; else string1="false";
                if (gt==1) string2="true"; else string2="false";
                if (eq==1) string3="true"; else string3="false";
                $display("%0d %0d\t%0s\t%0s\t%0s", A, B, string1, string2, string3);
                #3 A = 4'd4; B = 4'd6; #1;
                if (lt==1) string1="true"; else string1="false";
                if (gt==1) string2="true"; else string2="false";
                if (eq==1) string3="true"; else string3="false";
                $display("%0d %0d\t%0s\t%0s\t%0s", A, B, string1, string2, string3);
                #5 A = 4'd9; B = 4'd8; #1;
                if (lt==1) string1="true"; else string1="false";
                if (gt==1) string2="true"; else string2="false";
                if (eq==1) string3="true"; else string3="false";
                $display("%0d %0d\t%0s\t%0s\t%0s", A, B, string1, string2, string3);
                #7 A = 4'd10; B = 4'd1; #1;
                if (lt==1) string1="true"; else string1="false";
                if (gt==1) string2="true"; else string2="false";
                if (eq==1) string3="true"; else string3="false";
                $display("%0d %0d\t%0s\t%0s\t%0s", A, B, string1, string2, string3);
                #9 $finish;
        end
endmodule
```

Transcript

```
VSIM 44> run
# GetModuleFileName: The specified module could not be found.
#
#
# A B   A<B     A>B     A==B
# 0 0   false   false   true
# 7 7   false   false   true
# 4 6   true    false   false
# 9 8   false   true    false
# 10 1  false   true    false
# ** Note: $finish    : C:/intelFPGA_lite/20.1/Drill5_1.v(40)
#    Time: 30 ps  Iteration: 0  Instance: /drill5_1
# 1
# Break in Module drill5_1 at C:/intelFPGA_lite/20.1/Drill5_1.v line 40
```

Wave



Drill5_2

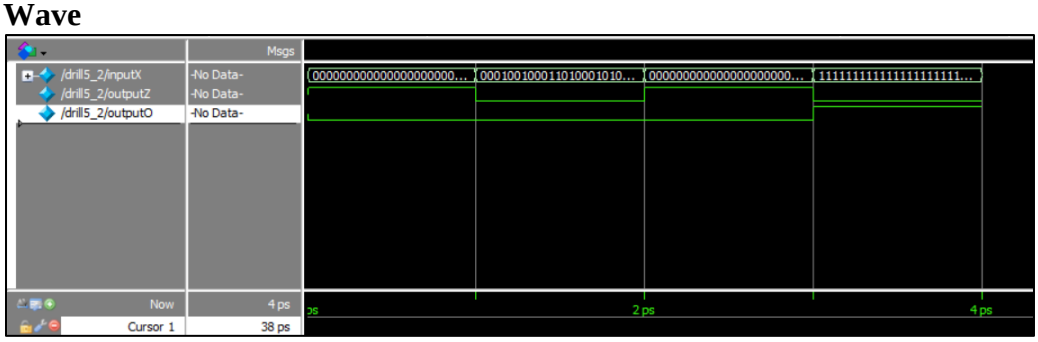
HDL Code

```
module AllBit(input [31:0]x, output zero, one);
        assign zero=~(|x);
        assign one=&x;
endmodule

module drill5_2;
        reg [31:0] inputX;
        wire outputZ, outputO;

        AllBit Abit(inputX, outputZ, outputO);
        initial fork
                $display("Time\tIn\t\tOut(1)\tOut(0)");
                $monitor("%0t\t%h\t\t%h\t%h",$time,inputX,outputO,outputZ);
                inputX=32'h0;
                #1 inputX=32'h12345678;
                #2 inputX=0;
                #3 inputX=32'hFFFFFFFF;
                #4 inputX=32;
                #4 $finish;
        join
endmodule
```

Transcript

```
VSIM 51> run
# GetModuleFileName: The specified module could not be found.
#
#
# Time  In              Out(1)  Out(0)
# 0     00000000        0       1
# 1     12345678        0       0
# 2     00000000        0       1
# 3     ffffffff        1       0
# ** Note: $finish    : C:/intelFPGA_lite/20.1/Drill5_2.v(19)
#    Time: 4 ps  Iteration: 0  Instance: /drill5_2
# 1
# Break in Module drill5_2 at C:/intelFPGA_lite/20.1/Drill5_2.v line 19
```

Wave



Drill5_3

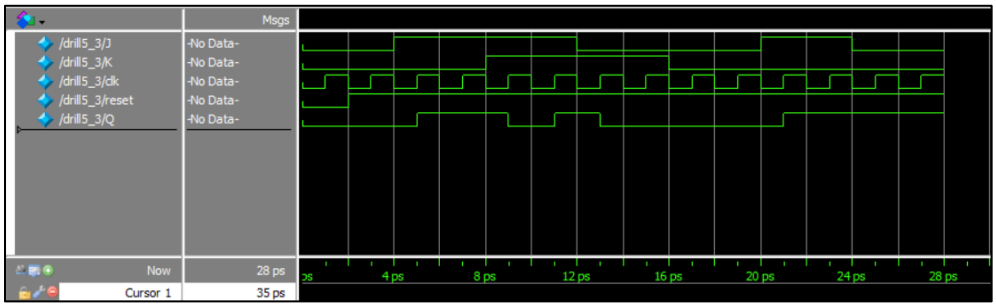
HDL Code

```
module JK_flipflop(output A, input J, K, clk, reset);
        wire AofJK;
        assign AofJK=(J&~A)|(~K&A);
        D_flipflop JKf(A,AofJK,clk, reset);
endmodule

module D_flipflop(output reg B, input D, clk, reset);
        always@(posedge clk, negedge reset)
        begin
                if(~reset)
                        B<=1'b0;
                else
                        B<=D;
        end
endmodule

module drill5_3;
        reg J, K, clk, reset;
        wire Q;
        always #1 clk=~clk;

        JK_flipflop JKF(Q,J,K,clk,reset);

        initial clk=0;
        initial reset=0;
        initial J=0;
        initial K=0;
        initial $monitor("clk=%b reset=%b, J=%b, K=%b, Q=%b",clk,reset,J,K,Q);

        initial fork
                #28 $finish;
                #2 reset=1;
                #4 J=1;
                #8 K=1;
                #12 J=0;
                #16 K=0;
                #20 J=1;
                #24 J=0;
        join
endmodule
```

Transcript

```
VSIM 3> run
# clk=0 reset=0, J=0, K=0, Q=0
# clk=1 reset=0, J=0, K=0, Q=0
# clk=0 reset=1, J=0, K=0, Q=0
# clk=1 reset=1, J=0, K=0, Q=0
# clk=0 reset=1, J=1, K=0, Q=0
# clk=1 reset=1, J=1, K=0, Q=1
# clk=0 reset=1, J=1, K=0, Q=1
# clk=1 reset=1, J=1, K=0, Q=1
# clk=0 reset=1, J=1, K=1, Q=1
# clk=1 reset=1, J=1, K=1, Q=0
# clk=0 reset=1, J=1, K=1, Q=0
# clk=1 reset=1, J=1, K=1, Q=1
# clk=0 reset=1, J=0, K=1, Q=1
# clk=1 reset=1, J=0, K=1, Q=0
# clk=0 reset=1, J=0, K=1, Q=0
# clk=1 reset=1, J=0, K=1, Q=0
# clk=0 reset=1, J=0, K=0, Q=0
# clk=1 reset=1, J=0, K=0, Q=0
# clk=0 reset=1, J=0, K=0, Q=0
# clk=1 reset=1, J=0, K=0, Q=0
# clk=0 reset=1, J=1, K=0, Q=0
# clk=1 reset=1, J=1, K=0, Q=1
# clk=0 reset=1, J=1, K=0, Q=1
# clk=1 reset=1, J=1, K=0, Q=1
# clk=0 reset=1, J=0, K=0, Q=1
# clk=1 reset=1, J=0, K=0, Q=1
# clk=0 reset=1, J=0, K=0, Q=1
# clk=1 reset=1, J=0, K=0, Q=1
# ** Note: $finish    : C:/intelFPGA_lite/20.1/Drill5_3.v(31)
#    Time: 28 ps  Iteration: 0  Instance: /drill5_3
# 1
# Break in Module drill5_3 at C:/intelFPGA_lite/20.1/Drill5_3.v line 31
```

Wave


Msgs
/drill5_3/J
/drill5_3/K
/drill5_3/clk
/drill5_3/reset
/drill5_3/Q
-No Data-
-No Data-
-No Data-
-No Data-
-No Data-
Now
28 ps
Cursor 1
35 ps
ps
4 ps
8 ps
12 ps
16 ps
20 ps
24 ps
28 ps

Descriptive Explanation

Drill5_1

This Verilog code defines a `magcom` module that compares two 4-bit inputs, `A` and `B`, and outputs three signals: `lt` (less than), `gt` (greater than), and `eq` (equal). The `drill5_1` module is a testbench that initializes `A` and `B` to 0 and simulates various test cases by assigning different values to `A` and `B` over time. For each test case, it checks the comparison results (`lt`, `gt`, and `eq`) and assigns corresponding string values ("true" or "false") to `string1`, `string2`, and `string3`. It then displays the values of `A`, `B`, and the results using the `$display` system task.

Drill5_2

This Verilog code defines the `AllBit` module that takes a 32-bit input `x` and produces two outputs: `zero` and `one`. The `zero` output is `1` if all bits of `x` are `0` (checked using the OR operation followed by negation), and the `one` output is `1` if all bits of `x` are `1` (checked using the AND operation). The `drill5_2` testbench module initializes the input `inputX` and uses the `AllBit` module to generate the corresponding outputs `outputZ` and `outputO`. It uses `$monitor` to display the values of `inputX`, `outputO`, and `outputZ` at each time step.

Drill5_3

This Verilog code implements a JK flip-flop using a D flip-flop. The `JK_flipflop` module generates an intermediate signal `AofJK` based on the JK logic (`(J & ~A) | (~K & A)`) and then passes it to a `D_flipflop` to produce the output `A`. The `D_flipflop` module updates the output `B` on the rising edge of the clock or the falling edge of the reset signal, where the reset sets `B` to `0`. The testbench (`drill5_3`) initializes signals like `clk`, `reset`, `J`, and `K`, and uses `$monitor` to display these values and the output `Q` at each time step. The clock toggles every 1 time unit, and the testbench sets different values for `J` and `K` at specific time intervals, simulating the behavior of the JK flip-flop, with the simulation ending after 28 time units.

Exercise5_1

This Verilog code defines a `BitComparator` module that compares two 8-bit inputs, `A` and `B`, and outputs whether `A` is greater than (`A_GT_B`) or less than (`A_LT_B`) `B`. The `exercise5_1` module initializes two 8-bit registers `A` and `B` with various binary values and compares them using the `BitComparator`. For each pair of `A` and `B`, the code checks if the values are equal, or if one is greater or less than the other and stores the result in a string. The comparison results are displayed in binary format, with the remarks showing whether `A` is greater than, less than, or equal to `B`. The `#5` delays are used to ensure that the comparison results are displayed after the changes in `A` and `B`.

Exercise5_2

The given Verilog code defines a 4-input, 16-output decoder module named `decoder`. The `decoder` module takes five inputs (`x`, `y`, `z`, `w`, and `e`) and produces a 16-bit output (`d`), where each bit in `d` represents a unique combination of the input variables. The output bits are generated using a series of AND gates combined with negations of the inputs based on their binary values. The `exercise5_2` module is a testbench that initializes the inputs and runs a series of test cases to observe how the decoder responds to different combinations of the input values, displaying the results at each simulation step. The `$display` command is used to print the values of the inputs and the output vector `d` at each time step, with the simulation running for a few clock cycles before ending with `$finish`.